Chapter 4 · Dependencies, keys and normal forms

A library ledger records BookID, Title, PublisherID and PublisherAddress. BookID determines the title and publisher. PublisherID determines the publisher's address. Repeating the address for every book makes a correction easy to miss. We will use these rules to decide what identifies a row and whether splitting the ledger preserves its information.

Vocabulary. X determines Y, written X -> Y, means that any two permitted tuples agreeing on X also agree on Y. X and Y can each be sets of attributes. A functional dependency is a rule about permitted instances. It cannot be established merely because a small displayed table happens to satisfy it. A trivial dependency has Y contained in X. X+ is the attribute closure of X under the given dependencies. F+ is the set of dependencies implied by F.

R15 Attribute closure and implied dependencies

Start X+ with X. Whenever a dependency's entire left side is inside the current closure, add its right side. Repeat until nothing changes. X -> Y follows exactly when Y is contained in the final closure.

Example: A -> B, B -> C on ABCD gives A+ = ABC. D is still missing. AD+ = ABCD.

Worked example W15. For R(ABCD), F={A→B,B→C}, start A+={A}. A→B adds B; B→C adds C. No rule adds D, so A+={A,B,C}. AD+ starts {A,D}, then reaches all four attributes. Thus A→C follows, A→D does not, and AD is a superkey. A dependency's whole left side must already be present: knowing A alone does not activate AB→D. Armstrong's basic rules are reflexivity (a set determines its subsets), augmentation (add the same attributes to both sides), and transitivity (chain two determining rules). Closure repeatedly applies those consequences.

Check before moving on. Is every left-side attribute available before a rule fires? Stop only at a fixed point.

R16 Find all candidate keys

1. Put every attribute absent from all right sides into the compulsory set: none can be obtained from another attribute.
2. Compute its closure.
3. Add subsets of remaining attributes in increasing size, computing each closure.
4. Keep a set only if it determines the whole schema and is minimal.
5. Stop exploring any branch already containing a found candidate key.

With A -> B and B -> A on ABC, C is compulsory. Both AC and BC are candidate keys. Finding AC does not finish the question. No dependencies at all makes the whole schema the only candidate key.

Worked example W16. On ABCD, let A→B, B→A, C→D. C never appears on a right side, so every key must contain C. C+={C,D}, still missing A,B. AC+ and BC+ both equal ABCD. Neither A nor C alone is a key, and neither B nor C alone is a key. Candidate keys: AC,BC. Their union makes A,B,C prime; D is nonprime. A minimal key can be longer than another minimal key in a different schema. Enumerate all minimal choices, not only the first one found.

Check before moving on. Did you find every minimal key? Mark all prime attributes only after that search.

R17 Count superkeys

If a schema has n attributes and exactly one candidate key of size k, there are 2^{n-k} superkeys. With multiple keys, count the union of their supersets. Use inclusion-exclusion or enumerate small subsets. Do not add supersets counts without removing overlap.

For AC and BC on ABC: the superkeys are AC, BC and ABC, giving three. Summing two counts of two would count ABC twice.

Worked example W17. With keys AC and BC on ABCD, supersets of AC are AC,ABC,ACD,ABCD. Supersets of BC are BC,ABC,BCD,ABCD. The overlap is ABC,ABCD. Hence 4+4−2=6 superkeys. If an extra E can be obtained from A, the same keys have eight supersets each and four overlaps, giving 12. An extra unconstrained attribute instead becomes compulsory in every key. Counting optional attributes requires knowing which attributes the key already determines.

Check before moving on. Did you remove overlapping supersets? Does an added column become optional or compulsory?

R18 Minimal cover and equivalence

Split right sides into single attributes. Test each proposed extraneous left-side attribute by computing closure under the current dependency set. Remove it only if the dependency remains implied. To test a redundant dependency, remove that dependency and compute the left-side closure using the others. Repeat until no further reduction applies. A minimal cover need not be unique.

Two dependency sets are equivalent if every dependency in each set follows from the other. Test both directions. Comparing their written left sides is insufficient.

Worked example W18. F={A→BC,B→C,AB→D}. Split A→BC into A→B and A→C. In AB→D, B is extraneous because starting from A, A→B already supplies it; simplify to A→D. A→C is redundant because A→B→C still derives C after that rule is removed. A minimal cover is {A→B,B→C,A→D}. To check equivalence, test every original rule from the cover and every cover rule from the original set. Remove A→D from the cover and A+ loses D, so equivalence fails. Removing a rule while testing redundancy is essential.

Check before moving on. For redundancy, remove the tested rule before calculating closure. Check equivalence both ways.

R19 Highest normal form

Find all candidate keys and prime attributes first. Assume 1NF only if values meet the question's atomic-domain assumption.

- 2NF prohibits a nonprime attribute depending on a proper subset of a candidate key.
- 3NF requires, for every nontrivial implied X -> A with a singleton right side, that X be a superkey or A be prime.
- BCNF requires every nontrivial implied determinant to be a superkey.

When F is a cover, testing each dependency of F with singleton right sides suffices to check 3NF and BCNF. For 2NF, test partial dependencies using closures of proper subsets of candidate keys; they may not be written explicitly. Check BCNF first, then 3NF, then 2NF. One violation disproves a normal form. Single-attribute candidate keys automatically avoid partial-key dependencies. “No transitive dependency” is too imprecise to replace the 3NF definition.

Worked example W19. On ABC with AB→C and C→B, the candidate keys are AB and AC. Every attribute is prime. C→B violates BCNF because C+={B,C} omits A. It does not violate 3NF because B is prime. The highest form is 3NF. Contrast A→B,B→C on ABC: A is the sole key, B and C are nonprime, and B→C violates 3NF. It is still 2NF because a single-attribute key has no nonempty proper subset causing a partial dependency. To see a 1NF-only case, take ABC with AB→C and A→C: AB is the key, C is nonprime, and A is a proper part of the key determining C. Atomic cells do not by themselves guarantee 2NF.

Check before moving on. Show a violating determinant and nonprime dependent. Do not use the vague rule no transitive dependencies.

R20 Binary lossless decomposition

For a split into R1 and R2 whose union is R, compute C = R1 intersect R2. The split is lossless under F exactly when C determines all of R1 or all of R2. Compute C+ using F and test containment. The test concerns all permitted instances, not only the sample table.

For R(ABC), A -> B, the split AB/AC is lossless because A determines AB. With only A -> B, splitting AB/BC is not guaranteed lossless: B does not determine AB or BC. A join can invent combinations, called spurious tuples.

Worked example W20. For ABC with A→B, split into AB and AC. The intersection is A and A+ contains AB, so the split is lossless. Split instead into AB and BC: their intersection B determines neither component. Take legal original rows (a1,b1,c1),(a2,b1,c2). AB joined with BC also creates (a1,b1,c2),(a2,b1,c1). Those extra combinations are spurious. The original relation still satisfies A→B. If B→C were additionally required, this counterexample would be illegal and the AB/BC split would be lossless.

Check before moving on. Use the intersection's closure under the full FDs. A single successful sample join does not prove losslessness.

R21 Nonbinary losslessness

Do not apply the binary intersection test once to a three-way split and declare it done. Use a sequence of justified binary decompositions when that sequence yields the stated result; otherwise use the chase.

Make a row per component and a column per original attribute. Use the shared distinguished symbol a_j for an attribute present in a component, and a fresh b_ij otherwise. For each X -> Y, rows agreeing on every X column must agree on Y. Merge symbols, preferring a distinguished symbol when present, and repeat. A row entirely of distinguished symbols establishes losslessness. Termination without such a row means the decomposition is lossy under these FDs. Practise one complete worked tableau before using this recipe.

Worked example W21. For ABCD, F={A→B,B→C}, components AB,BC,AD, make a chase tableau. Row AB is (aA,aB,b13,b14); row BC is (b21,aB,aC,b24); row AD is (aA,b32,b33,aD). A→B makes rows AB and AD agree on B, so b32 becomes aB. Now all three rows agree on B. B→C makes their C symbols aC. Row AD becomes (aA,aB,aC,aD), so the decomposition is lossless. Without B→C, its C remains b33 and no row becomes entirely distinguished. The chase then establishes that losslessness is not guaranteed. A convenient binary derivation gives the same positive result: split ABCD into ABC/AD using A→BC, then ABC into AB/BC using B→C.

Check before moving on. Write the full tableau and symbol changes. One entirely distinguished row is the success criterion.

R22 Dependency preservation

Project F+ onto each component: retain dependencies using only its attributes. Include implied local dependencies, not just original dependencies that happen to fit. Let G be their union. For each original X -> Y, compute X+ under G. The decomposition preserves dependencies exactly when all these tests succeed. Projected dependencies are already implied by F, so the reverse inclusion follows.

Lossless and dependency-preserving are different requirements. Check them separately.

Worked example W22. With A→B,B→C on ABC, the split AB/AC is lossless because A determines AB. Projected rules include A→B in AB and the implied A→C in AC. But under their union, B+={B}; B→C cannot be checked locally. It is not dependency-preserving. Split into AB/BC instead: the local rules are A→B and B→C, so all original rules remain enforceable, and B determines BC, making the split lossless too. Projection means rules from F+, not merely the written rules that fit.

Check before moving on. Include implied local rules. Losslessness and preservation are separate answers.

R23 Construct a decomposition

For BCNF, choose a violating X -> Y inside the current relation and split it into X union Y and the original attributes minus (Y minus X). Recompute projected dependencies and repeat. These splits are lossless; preservation is not guaranteed.

For a standard 3NF synthesis, find a minimal cover, make relations for its determinants and right sides, remove contained relations, and add a relation containing a candidate key if none already does. This construction gives a lossless dependency-preserving 3NF decomposition under functional dependencies. Explain those guarantees separately from whether a particular output also happens to be BCNF.

Worked example W23. Start ABC with AB→C,C→B. The BCNF violation C→B gives components BC and AC. Their common C determines BC, so the split is lossless. The projected local rules cannot derive AB→C, so preservation is lost. For 3NF synthesis, the cover produces ABC and BC; remove contained BC and keep ABC. ABC contains a candidate key, so no extra key relation is needed. It is 3NF and preserves the rules, although it is not BCNF. A decomposition's guarantees must be checked separately: a cleaner component is not proof of preserved constraints.

Check before moving on. For synthesis, check containment removal and whether some component contains a candidate key.

Previous-year questions, worked

Read the original question and options in the linked official paper. These are original explanations, not replacement question statements. PDF page numbers count from the first page of the downloaded file.

DA-2024-Q46 · Infer FDs

Official question, PDF page 29. Published key: **A; B; D**.

Given U→V,U→W,V→X,WX→Y,WX→Z, (VW)+ starts V,W, adds X from V, then Y,Z from WX. Thus VW→YZ and VW→Y hold. WX→YZ follows directly from the two given rules. No rule adds U to (VW)+, so VW→U fails. Correct options: A,B,D.

DA-2025-Q16 · Why preservation matters

Official question, PDF page 11. Published key: **C**.

If a constraint cannot be checked using component relations' local dependencies, information from components may need to be joined to test it. The intended operator is join, C. This is a statement about checking dependencies after decomposition; it does not assert that every decomposed query requires a join.

DA-2025-Q57 · All keys then BCNF

Official question, PDF page 31. Published key: **B; D**.

A+ gets B,C,E,F, then E gets D,G: all attributes. (BC)+ gets A and hence everything. E+={E,D,G} is not a key. Any key without A must get A through BC→A, so BC is the other minimal choice. E→D violates BCNF since E is not a superkey. A is not the only key and E is not a key. Thus B,D.

DA-2026-Q17 · Candidate-key enumeration

Official question, PDF page 12. Published key: **A**.

For A→BC,CD→E,E→A, D never appears on a right side, so every key contains D. AD obtains BC and then E. CD obtains E,A,B. ED obtains A,B,C. Each pair is minimal. BD cannot obtain A,C,E. The complete candidate-key set is AD,CD,ED, matching A.

CS-2024-set1-Q44 · Dependency rules

Official question, PDF page 28. Published key: **B; C; D**.

A wrongly deletes Y from the determinant. Rows (X=1,Y=1,Z=0,W=0) and (X=1,Y=2,Z=1,W=1) satisfy XY→ZW but violate X→ZW. B follows by splitting a right side. For C, XW supplies Y via W→Y, then XY supplies Z. D is transitivity. Correct: B,C,D.

CS-2024-set2-Q56 · Count possible disjoint FD sides

Official question, PDF page 37. Published key: **50 to 50**.

Each of four attributes has three choices: left side, right side, neither. Start with 3^4. Subtract 2^4 choices with empty left and 2^4 with empty right. The all-neither choice was subtracted twice, so add 1. Result: 81−16−16+1=50. These are possible nonempty disjoint syntax choices, not a claim that all 50 hold in one database.

CS-2025-set1-Q47 · BCNF and a lossless split

Official question, PDF page 47. Published key: **B; C**.

Name determines city and owner and is the original candidate key. All given nontrivial determinants are superkeys, so the original is BCNF and therefore also 3NF: A,D are false. Each component has name as a key, so B is true. The intersection name determines both components, so the split is lossless: C is true.

CS-2025-set2-Q46 · Partial and transitive violations

Official question, PDF page 35. Published key: **C; D**.

For R1(ABCDE), A and D cannot be derived from other attributes, so AD is the candidate key. D→E determines a nonprime attribute from a proper part of that key, violating 2NF and hence 3NF. For R2(ABCD), C is the candidate key since C→A→B,D. A→B and A→D have a non-superkey determinant and nonprime dependents, violating 3NF. The statements identifying both as not 3NF are C,D. A single-attribute key avoids 2NF violations, not 3NF violations.

CS-2022-Q14 · A convention-sensitive normal-form claim

Official question, PDF page 12. Published key: **A**.

The official answer is A. Under the usual exam convention excluding nontrivial empty determinants, a binary relation's only possible nontrivial FD is one attribute determining the other; that determinant is a key. However, if ∅→A is allowed on AB, ∅ need not determine B, so a binary relation need not be BCNF. Do not teach A as unconditional. B is false: AB→C,C→B gives all-prime attributes but violates BCNF. C is false because a relation can have all attributes prime. D is false because BCNF splitting need not preserve dependencies, as W23 shows.

CS-2022-Q31 · Count superkeys

Official question, PDF page 22. Published key: **8 to 8**.

With AB→C,BC→D,C→E on ABCDE, neither A nor B occurs on a right side, so both are compulsory. AB+ obtains C,D,E, giving the unique candidate key AB. C,D,E are optional additions to a superkey. There are 2^3=8 superkeys.

Chapter practice

Original questions. Write the answer and the intermediate reasoning. Each lettered part belongs to the same problem. The separate solutions guide contains hints, steps and a final check.

P15 · Closure

R(UVWXYZ) has U→V,U→W,V→X,WX→Y,WX→Z. (a) Compute (VW)+ and U+. Does VW→U follow? (b) Remove V→X. Recompute (VW)+ and explain which step stops working.

P16 · All keys

R(ABCD) has A→B,B→A,C→D. (a) Find all candidate keys and prime attributes. (b) Remove B→A and repeat. Show minimality, not only full closure.

P17 · NAT plus variation

Use R(ABCD) with A→B,B→A,C→D. (a) Count its superkeys. (b) Add attribute E and FD A→E; count again. What if E were added without any rule determining it?

P18 · Cover and equivalence

F={A→BC,B→C,AB→D}. (a) Find a singleton-right-side minimal cover. Check whether G={A→B,B→C,A→D} is equivalent to F. (b) Remove A→D from G. Give a closure witnessing the loss of equivalence.

P19 · Normal form

Assume atomic values and no nontrivial empty-determinant FDs. (a) R(ABC) has AB→C,C→B. Find all keys and its highest normal form. (b) Replace the FDs with A→B,B→C. Repeat. Give a specific violation of the next higher normal form.

P20 · Losslessness

R(ABC) has only A→B. (a) Test AB/AC and AB/BC for losslessness. Give a two-row counterexample for the failing split. (b) Add B→C. Does AB/BC become lossless?

P21 · Chase

R(ABCD), F={A→B,B→C}, is split into AB,BC,AD. (a) Build a chase tableau and decide losslessness. (b) Remove B→C and repeat. State the row or missing distinguished symbol that decides each result.

P22 · Preservation

R(ABC) has A→B,B→C. (a) Does AB/AC preserve all dependencies? List its implied local FDs. Is it lossless? (b) Repeat for AB/BC. Use B's closure under projected rules to explain the difference.

P23 · Construct and check

R(ABC) has AB→C,C→B. (a) Perform the BCNF split using C→B. Check each component's normal form, losslessness and preservation. (b) Perform 3NF synthesis from these FDs, remove contained relations and check whether a key relation must be added. Why can keeping ABC be acceptable?

Before the exam

Find every key. Check losslessness and preservation separately. Include implied local dependencies. Mark assumptions.